

4. 시연 4개 요약

발표 25분 동안 실행된 4개 시연의 화면별 가이드. 실행 명령 + 화면 + 메시지를 시연당 1쪽으로 압축.

시연 1 — write-back

실행 명령

```
$ ./run.sh db/check.py
$ ./run.sh inference/run_inference_demo.py
```

출력 화면

💡 **화면 약어:** EMD_ = 읍·면·동 행정구역 (예: EMD_여수_상암동 = 여수시 상암동),
S= = 종합 위험 점수 (0~1)

```
$ ./run.sh db/check.py

[검증] source 적재: REAL 18 / MOCK 6 / EXCLUDE 5
[검증] segment 적재: 5건

$ ./run.sh inference/run_inference_demo.py

[추론] 5 segment 점수+State 산출 → 그래프 write-back

EMD_여수_상암동   S=0.534   PriorityPreWatering
EMD_장흥_유치면   S=0.454   NotActionable
EMD_나주_봉황면   S=0.315   EnhancedMonitoring
EMD_무안_운남면   S=0.315   EnhancedMonitoring
EMD_광주_동구     S=0.180   GeneralManagement
```

✅ 그래프에 저장 완료

핵심 메시지

4주차 source 29건 위에 segment 5건이 올라가고, Python 이 그래프의 임계값을 읽어 점수를 계산한 다음 그 결과를 그래프에 **다시 써넣었다**. 다음 시연부터는 전부 이 저장된 결과를 조회한다.

🔥 **메시지 ①**: "추론 결과가 다시 데이터가 된다 (write-back)"

시연 2 — fun 듀얼 ★

실행 명령 (터미널 2개 분할)

```
[왼쪽] $ ./run.sh tests/test_param_externalization.py
[오른쪽] $ ./run.sh tests/test_python_vs_fun.py
```

출력 화면

[왼쪽: 정책 시뮬레이션]

```
v1 임계값: 0.20/0.40/0.60/0.80
v2 임계값: 0.15/0.30/0.50/0.70

★ EMD_나주_봉황면 0.315
  v1: 감시 강화 → v2: 검토 격상 ↑

→ 코드 수정 0줄, INSERT로 시뮬레이션
```

[오른쪽: 회귀 검증]

```
회귀 검증 — Python ↔ fun 일치

[v1 임계값] 5/5 일치 ✓
[v2 임계값] 5/5 일치 ✓

→ 10/10 통과
"추론을 fun으로 옮겨도 결과 동일"
```

핵심 메시지

왼쪽은 **코드 수정 없이 정책 노드만 바꿔** 시뮬레이션한 결과. 나주 봉황면이 격상됐다.

오른쪽이 진짜 새로운 부분 — 같은 추론을 TypeDB **fun** 이 **직접 수행**했는데 결과가 100% 일치. 즉 추론을 그래프 안으로 완전히 옮길 수 있다.

🔥 **메시지 ②**: "정책 변경 = INSERT 한 줄, 코드 0줄"

🔥 **메시지 ⑥**: ★ "그래프가 직접 추론할 수 있다"

시연 3 — chat.py + advisory

실행 명령

```
$ ./run.sh llm/chat.py
```

출력 화면 (질의 3건)

Q1: EMD_여수_상암동 위험도 알려줘

💬 답변: "여수 상암동은 현재 PriorityPreWatering(우선 예비주수) 상태입니다.
기상특보 발효로 인해 단계가 격상되었습니다."

Q2: MOCK source 모두 알려줘

💬 답변: "[리스트 나열]... ⚠️ 일부 데이터는 추정값(MOCK)입니다."
(advisory 자동 부착)

Q3: 1990년 1월 광주 동구 위험도?

💬 답변: "해당 시점의 데이터가 존재하지 않습니다."
(실패 처리 — 추측하지 않음)"

핵심 메시지

Q1 — 일반 질의. DB 조회 결과를 자연어로 풀어준다.

Q2 — MOCK 인용 시 **advisory 자동 부착**. source.availability-status 가 답변까지 따라가는 구조.

Q3 — 환각 방지의 본질. **DB 에 없는 데이터를 LLM 이 추측해 답하지 않는다**. 명확히 거부한다.

🔴 메시지 ③: "감사 정보가 자연어에 묻어 나온다"

🔴 메시지 ④: "DB 에 없으면 LLM 도 답을 만들지 않는다"

시연 4 — 평가셋 14건

실행 명령

```
$ ./run.sh llm/eval/run_eval.py
```

출력 화면 (실시간 카운트)

평가셋 14건 자동 실행 중...

[3/14] 단순

↺ 재시도 1회 → 성공

← "자가 수정!"

[11/14] 복잡

↺ 재시도 1회 → 성공

[12/14] 복잡

✖ 1990년 데이터 없음 (의도된 거부)

✓ 직접 성공

8건

↺ 자가 수정

3건 ← 핵심 관찰 포인트

↺ Fallback

0건

✖ 완전 실패

3건 ← 1건은 환각 방지용 데이터 부재

직접 처리율: 11/14 (79%)

핵심 메시지

↺ 표시가 뜰 때마다 LLM 이 한 번 틀렸다가 시스템이 다시 시도해서 성공한 것이다. 이게 Self-correcting 이다.

✖ 는 데이터가 없는 경우로, 시스템이 거짓말을 하지 않고 정확히 거절한 결과다.

🔥 메시지 ⑤: "자가 수정 + 정량 측정 = 측정 가능한 LLM 시스템"

6대 Takeaway 총정리

#	메시지	시연
①	write-back	1
②	외부화 (코드 0출)	2
③	advisory	3
④	환각 방지	3·4
⑤	Self-correcting	4
⑥	★ 자족 추론	2

4개 시연이 함께 증명하는 것

"사실은 그래프와 함수가, 표현은 LLM 이. 정책은 노드로 외부화. 측정·자가수정은 시스템이."

- 시연 1:2 → "사실은 그래프와 함수가" 부분 증명
- 시연 3 → "표현은 LLM 이 + 정책은 노드로" 부분 증명
- 시연 4 → "측정·자가수정은 시스템이" 부분 증명

시연을 직접 재현하고 싶다면

받으신 코드 zip 안에 모든 시연이 들어 있다. 환경 셋업 + 명령 시퀀스는 [5_코드_안내.md] 참고.

한 줄 요약

"4개 시연 = 4개 실행 명령 + 4개 출력 화면 + 6개 메시지. 한 가설(사실/표현 분리)의 4각도 증거."